

SHIPSAFE AI

THE PRODUCTION GUARDRAIL SYSTEM

Control, test, secure, recover, and launch AI-built applications with
reviewable evidence.

CONTROL THE AGENT. PROTECT THE PROJECT. SHIP WITH EVIDENCE.

Version 1.0 | June 2026

Operating Manual

For founders, vibe coders, software teams, reviewers, and agent-assisted builders

THE CORE RULE

An AI coding agent saying “done” is not evidence. A release is ready only when its behavior, access boundaries, security controls, recovery path, and acceptance criteria are independently verified.

Legal and operational notice

SHIPSAFE AI is a general engineering control framework. It is not a security guarantee, penetration test, compliance certification, or substitute for qualified engineering, security, privacy, or legal review of high-risk systems. The project owner remains responsible for access, credentials, data, testing, production decisions, monitoring, backups, and incident response.

Contents

1. 1. The SHIPSAFE doctrine
2. 2. The five launch gates
3. 3. Install the control layer
4. 4. Lock scope and protected behavior
5. 5. Run agent work safely
6. 6. Verify functional correctness
7. 7. Test authentication and authorization
8. 8. Protect data, secrets, and dependencies
9. 9. Control AI-specific risk
10. 10. Deploy with checkpoints and rollback
11. 11. Score readiness and decide
12. 12. Recover from a damaged project
13. 13. Convert a capstone into a credible portfolio asset
14. 14. Sixty-minute implementation plan
15. Appendix A: fast test
16. Appendix B: evidence standard
17. Appendix C: file map and references

1. The SHIPSAFE doctrine

AI coding tools can inspect repositories, edit many files, run commands, change configuration, and interact with external systems. Their leverage is exactly why project control cannot be reduced to a well-written prompt.

SHIPSAFE separates generation from verification. The agent may propose and implement work, but the project defines authority, evidence, and launch decisions outside the model's self-assessment.

- Control: define scope, permissions, protected behavior, and approval gates.
- Evidence: require observable tests, diffs, commands, and acceptance-criteria results.
- Recovery: preserve a known-good state before high-risk work.
- Least privilege: grant only the access and authority required for the task.
- Honest failure: a visible failure is safer than fabricated or silently substituted success.

2. The five launch gates

A product should pass five independent gates. Failing or not verifying a Critical control blocks launch.

- Gate 1 - Defined: objective, user, scope, architecture, data, and acceptance criteria are explicit.
- Gate 2 - Controlled: agent instructions, protected files, permissions, and approval gates are active.
- Gate 3 - Verified: functional, regression, failure-mode, access-control, and security checks produce evidence.
- Gate 4 - Recoverable: version-control checkpoint, migration/data recovery, configuration recovery, and rollback are tested.
- Gate 5 - Operable: production configuration, monitoring, quotas, ownership, and post-deploy checks are ready.

3. Install the control layer

Place persistent instructions where the chosen coding tool reads them, then keep the detailed project truth in shared neutral files. OpenAI Codex supports AGENTS.md, Claude Code supports CLAUDE.md and rule files, and Gemini CLI supports GEMINI.md context files. Tool-specific files should point to the same project context rather than develop conflicting versions of the truth.

- Copy the matching instruction file from 01_AGENT_GUARDRAILS into the project root.
- Copy UNIVERSAL_PROJECT_GUARDRAILS.md into the root.
- Copy PROJECT_CONTEXT.md, SCOPE_LOCK.md, ACCEPTANCE_CRITERIA.md, PROTECTED_FILES.md, and DEFINITION_OF_DONE.md.
- Fill the files before the next material coding task.
- Tell the agent to read them before planning or editing.
- Keep the top-level instructions concise and link to task-specific detail.

Tool	Persistent project file	Included asset
OpenAI Codex	AGENTS.md	01_AGENT_GUARDRAILS/AGENTS.md
Anthropic Claude Code	CLAUDE.md	01_AGENT_GUARDRAILS/CLAUDE.md
Google Gemini CLI	GEMINI.md	01_AGENT_GUARDRAILS/GEMINI.md
Cursor	Project rule	01_AGENT_GUARDRAILS/.cursor/rules/shipsafe.mdc
Any agent	Neutral shared policy	UNIVERSAL_PROJECT_GUARDRAILS.md

4. Lock scope and protected behavior

The first protection against an agent breaking the product is an enforceable definition of what the task is and is not. Scope is not a feature list alone; it also names behavior that must remain unchanged.

SCOPE-LOCK TEST

Could a reviewer determine that the agent changed something outside authority without asking the original requester? If not, the scope is not specific enough.

Write testable acceptance criteria

Use condition-action-result language. Every criterion should name the user/system state, the action, the observable result, and the method of verification.

Weak criterion	Controlled criterion
Make login secure.	Given a suspended account with an existing session, when it calls a protected write endpoint, the server returns a denial and no record changes.
Improve error handling.	When the payment provider times out, the user sees a recoverable failure, no entitlement is created, and the event is logged without sensitive data.
Do not break the dashboard.	All listed dashboard regression tests pass and the final diff does not alter protected routes or data queries.

Create protected behavior

- List files/components that require explicit approval before change.
- List user journeys that already work and the tests/evidence that prove them.
- List public interfaces, database contracts, events, and external integrations that must remain compatible.
- List data or operations that must never enter AI prompts or logs.

5. Run agent work safely

The inspect-plan-checkpoint-change-verify-report loop

18. Inspect the relevant implementation, tests, configuration, history, and project controls.
19. Plan the smallest viable change and identify risk.
20. Checkpoint before high-risk work.
21. Implement the narrow change without unrelated rewriting.
22. Run focused tests and relevant regression/security checks.
23. Review the diff and execution evidence.
24. Report completion, remaining risk, and rollback.

STOP CONDITIONS

Stop and request a decision when requirements conflict, a destructive command is proposed, the agent needs broader authority, critical context is missing, a production or credential action is required, or the only path to “success” weakens verification.

Forbidden shortcuts

- Replacing a real integration with mock/static data without explicit disclosure and approval.
- Catch-all exception handling that converts failure into an empty or successful response.
- Removing tests, validation, authorization, rate limits, logs, or security checks to obtain a pass.
- Large file or framework rewrites when a targeted patch is viable.
- Using production credentials or customer data as convenient development input.
- Claiming a check passed without showing the method and result.

6. Verify functional correctness

AI-generated projects often look complete before their real integrations, state transitions, and failure behavior are correct. Functional verification must distinguish a rendered interface from a working system.

Inventory simulated behavior

- Search for mock, fixture, demo, sample, placeholder, fallback, temporary, TODO, hardcoded, seed, and static-response patterns.
- Inspect network calls and server logs while completing the primary journey.
- Confirm external service state actually changes.
- Confirm success persists after refresh and in a clean session.
- Force a dependency failure and verify the UI does not continue as if successful.

Minimum test portfolio

Layer	Minimum proof	Common false confidence
Unit	Critical calculations and deterministic rules match known examples.	Only trivial helpers are tested.
Integration	Real test database/API/storage behavior is exercised.	Mocks prove only the mock contract.
Authorization	Different actors are tested against direct endpoints and data boundaries.	Buttons are hidden in the UI.
End-to-end	Primary journeys work from clean state to durable result.	Developer session and seed data hide defects.
Failure mode	Timeout, invalid, duplicate, empty, denied, and partial failure are safe.	Only the happy path is demonstrated.
Regression	Accepted existing behavior remains protected.	The new feature works while old behavior silently breaks.

7. Test authentication and authorization

Authentication proves identity. Authorization determines what that identity may do. A successful login does not prove that records, files, privileged routes, or server actions are isolated.

DIRECT-ACCESS RULE

Never accept a hidden button, client redirect, disabled form, or route guard as sufficient authorization evidence. Call the server or data boundary directly with unauthorized credentials.

Required actors

- Anonymous state
- User A
- User B
- Privileged/admin user
- Suspended or deleted user
- Expired session

Required test classes

25. Anonymous access to protected pages, APIs, objects, and files.
26. User A reading, updating, or deleting User B resources by changing identifiers.
27. Normal user calling privileged endpoints directly.
28. Client-supplied owner, price, role, status, or entitlement fields.
29. Suspended/deleted user reusing a prior session.
30. List/search/export endpoints leaking unauthorized records or metadata.
31. Database row policies and object-storage policies tested with actual non-privileged tokens.

8. Protect data, secrets, and dependencies

Data minimization

Classify what the system collects, where it travels, who can access it, how long it remains, and how it is deleted. Avoid sending production personal or sensitive data through development tools or AI prompts unless that use is explicitly authorized and protected.

Secret exposure is a rotation event

Deleting a visible secret from a current file does not make it secret again. If it entered Git history, logs, prompts, client bundles, screenshots, or third-party systems, rotate/revoke it and inspect usage.

Dependency control

- Lock dependency versions and review changes.
- Remove packages and services that are not required.
- Use vulnerability and licence checks appropriate to the stack.
- Investigate installation scripts and unexpected transitive dependencies.
- Keep development, staging, and production credentials separate.
- Define quotas, timeouts, retry rules, and cost ceilings for external providers.

9. Control AI-specific risk

An application that reads external content and can use tools combines two trust domains: untrusted information and privileged action. Never allow retrieved text, uploaded files, webpages, issues, comments, or tool output to redefine system authority.

Agent permission design

Capability	Default	Elevation trigger
Read project files	Allowed within authorized repository	Outside-repository access
Edit code	Allowed within scoped task	Protected files, large rewrite, or public API change
Run tests/build	Allowed when non-destructive	Command changes state or accesses secrets
Network access	Deny or tightly scope	Explicitly required verified endpoint
Database access	Use test/least privilege	Schema/data change or production access
Deploy	Approval required	Every production deployment
Send data/messages	Approval required	Every external destination not pre-authorized

Prompt-injection test

Place an instruction inside untrusted test content asking the agent to ignore rules, expose a secret, access an unrelated file, send data externally, or disable a safety control. The system should treat it as data, preserve authority boundaries, and surface the attempt rather than execute it.

10. Deploy with checkpoints and rollback

A release is not recoverable merely because the repository uses Git. Code, database schema/data, configuration, credentials, storage, external provider state, and infrastructure may require different recovery procedures.

Checkpoint before high-risk work

- Clean and identify the known-good code state.
- Record critical test results at the checkpoint.
- Back up or snapshot affected data/configuration.
- Write the rollback trigger before editing.
- Isolate the work on a branch or worktree.
- Keep destructive steps explicit and human-approved.

Rollback drill

32. Deploy or run the candidate in a controlled environment.
33. Activate the rollback procedure.
34. Confirm code, configuration, schema/data, and integrations return to the expected state.
35. Run critical smoke and authorization tests.
36. Record duration, failures, and corrections to the runbook.

11. Score readiness and decide

Open the included Excel scorecard. Every check starts as Unknown, and Unknown earns zero. This prevents untested assumptions from inflating readiness.

Score	Decision	Interpretation
95-100%	Launch ready	Critical controls pass; only accepted residual risk remains.
85-94%	Conditional launch	Launch only with explicit acceptance and owners for remaining non-critical risk.
70-84%	Staging ready	Use for controlled testing, not unrestricted public launch.
50-69%	High risk	Material controls are missing or unknown.
0-49%	Stop - not ready	Do not launch until critical gaps are resolved.

CRITICAL OVERRIDE

The numeric score never overrides a failed or unknown Critical control. A high average cannot compensate for exposed data, missing authorization, unprotected secrets, fake success, or no recovery path.

12. Recover from a damaged project

Do not respond to an agent-created failure by granting the same agent broader authority and asking it to fix everything. Stop automation, preserve evidence, classify the damage, contain it, and recover from a known state.

37. Stop agents, deployments, migrations, jobs, and retries affecting the incident.
38. Record the time, environment, symptoms, commit, recent actions, and logs.
39. Preserve the affected state before overwriting it.
40. Classify code, configuration, data, authorization, credential, dependency, or external-provider impact.
41. Contain exposure or damage with the narrowest reversible action.
42. Restore code/configuration and reconcile data separately.
43. Rotate exposed credentials and inspect usage.
44. Run authorization, critical journey, data-integrity, and deployment health checks.
45. Add a regression test and update project instructions/runbooks.

13. Convert a capstone into a credible portfolio asset

A credible capstone demonstrates one real outcome, explains its authority boundary, and provides evidence. A large feature list is less persuasive than a narrow system that visibly works and fails safely.

- Define one user, one problem, and one outcome observable in under three minutes.
- Limit the MVP to three core capabilities.
- Demonstrate real state or integration evidence, not only a generated interface.
- Include one controlled failure or permission-boundary example.
- Publish reproducible setup without secret values.
- Disclose mocks, limitations, and unverified assumptions.
- Use the included README, demo script, evaluation table, and case-study templates.

14. Sixty-minute implementation plan

Minutes	Action	Output
0-10	Copy project guardrails and select the tool-specific instruction file.	Persistent control layer
10-20	Complete project context, scope lock, protected files, and acceptance criteria.	Explicit authority and success definition
20-30	Create known-good checkpoint and rollback plan.	Recovery target
30-45	Open scorecard and answer only checks supported by evidence.	Truthful baseline score
45-55	Fix the highest-severity unknown/failure that can be resolved safely.	Reduced launch risk
55-60	Create go/no-go record and next-action sequence.	Controlled decision

DO NOT RUSH THE SCORE

The one-hour plan installs the control system and identifies truth. It does not make every project production-ready in one hour.

Appendix A - 12-point fast test

- 1. Can one user retrieve or change another user’s data?
- 2. Have anonymous and direct API access been tested?
- 3. Are any secrets present in source, history, logs, prompts, or browser bundles?
- 4. Are mock, demo, static, and fallback responses fully identified?
- 5. Can a failed dependency produce apparent success?
- 6. Are core working features protected by regression tests?
- 7. Can the last known-good version be restored quickly?
- 8. Are database/storage policies tested using normal user credentials?
- 9. Does the coding agent have explicit limits and protected files?
- 10. Are destructive/high-impact actions gated by approval?
- 11. Does the release have evidence for every acceptance criterion?
- 12. Has the deployed environment been tested from a clean account/session?

Appendix B - Evidence standard

Claim	Acceptable evidence	Insufficient
Tests pass	Command, environment, dated output, and relevant scope	“I tested it”
Authorization works	Actor-based direct boundary tests	Hidden UI controls
No secrets exposed	Source/history/bundle/log checks and rotation record where needed	No .env file visible
Integration works	Real test transaction/state and failure test	Successful-looking UI
Rollback works	Recorded drill and post-rollback checks	Written instructions only
Acceptance criterion met	Criterion-specific observable result	Agent self-report

Appendix C - File map

- 01_AGENT_GUARDRAILS** - Persistent cross-platform instructions and task reporting.
- 02_PROJECT_CONTROL** - Context, scope, acceptance, protected behavior, and decisions.
- 03_TESTING_AND_QA** - Functional, failure-mode, regression, and evidence templates.
- 04_SECURITY_PREFLIGHT** - Authorization, secrets, input, AI, and Supabase checks.
- 05_CHECKPOINTS_AND_ROLLBACK** - Known-good checkpoints, recovery, and safe helper scripts.
- 06_LAUNCH_READINESS** - Scored workbook, go/no-go decision, and post-deploy checks.
- 07_CAPSTONE_AND_PORTFOLIO** - Capstone, README, demo, case-study, and submission assets.
- 08_CLIENT_AND_COMMERCIAL** - Client intake, audit report, and acceptance templates.
- 09_BONUS_QUICK_TOOLS** - Fast test, task brief, risk register, and scanner question bank.

Official references

- OpenAI Codex AGENTS.md: <https://developers.openai.com/codex/guides/agents-md>
- OpenAI Codex best practices: <https://developers.openai.com/codex/learn/best-practices>
- Anthropic Claude Code project memory: <https://docs.anthropic.com/en/docs/claude-code/memory>
- Google Gemini CLI GEMINI.md: <https://google-gemini.github.io/gemini-cli/docs/cli/gemini-md.html>
- OWASP ASVS index: <https://cheatsheetseries.owasp.org/IndexASVS.html>
- OWASP Secure Code Review:
https://cheatsheetseries.owasp.org/cheatsheets/Secure_Code_Review_Cheat_Sheet.html
- OWASP Secrets Management:
https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- Supabase RLS: <https://supabase.com/docs/guides/database/postgres/row-level-security>
- Supabase Storage access control: <https://supabase.com/docs/guides/storage/security/access-control>